ACADEMIC PROJECT · REVIEW 3 (FINAL)

# Design and Implementation of a Database Management System for Bank Account and Transaction Management System

Project 55 · Working application on the normalized MySQL database


PRESENTED BY
M. Srilakshmi Reddy

ROLL NUMBER
25WU0102140

PROGRAM
AI-ML · Woxsen University


ACADEMIC YEAR
2026 – 27

# Review 3 — What Has Been Completed

| 14 | 3 | 2 | 7 | 16 |
|---|---|---|---|---|
| Tables in 3NF | Triggers | Stored procedures | Reporting views | Test cases, all pass |

### REVIEW 2 → REVIEW 3

- Review 2 delivered the **normalized schema, constraints, triggers, procedures, sample data, views and queries**.
- Review 3 puts a **role-based web application** on that unchanged database, so every screen reads and writes the real tables.
- The application adds only one table of its own, `staff_user`, for staff logins.

### EVIDENCE IN THIS REVIEW

- Live demo in the browser, and the full Node.js + MySQL version.
- Screenshots captured from the running app and the MySQL console.
- End-to-end test suite: **15 of 15** passing on a freshly loaded database.
- Final report (36 pages) and the repository.

# System Architecture




- The database is the **single source of truth**: balances, ledger and loan state are computed by triggers and procedures, not by the browser or the API.
- Every query uses **prepared statements**, so user input never becomes SQL text.
- Multi-step actions (transfer, loan, repayment) run inside **one transaction** and roll back on any error.
- The API only maps database errors to clear HTTP codes: 422, 409, 400, 403.

# Technology Stack and Design Choices

| LAYER | CHOICE | WHY |
|---|---|---|
| **Database** | MySQL 8, InnoDB | Transactions, foreign keys, CHECK, triggers, stored procedures |
| **Back end** | Node.js 18+, Express 5, mysql2 | Small, readable REST API; connection pool; promise API |
| **Authentication** | JWT in an HTTP-only cookie, bcryptjs | Passwords hashed; token not readable by page scripts |
| **Front end** | Plain HTML, CSS, JavaScript (ES modules) | No build step, no CDN; fonts bundled; opens anywhere |
| **Access control** | `lib/permissions.js` | One table of permissions per role, shared with the UI |
| **Testing** | `npm test` (end-to-end, real database) | Checks rules through the API, including rollback and parallel withdrawals |

# Application Modules and Database Mapping

| MODULE | PURPOSE | TABLES AND VIEWS USED |
|---|---|---|
| **Dashboard** | Totals, branch summary, latest postings | v_branch_summary, v_loan_outstanding, transaction |
| **Customers** | Register and search customers | customer, account_holder |
| **Accounts** | Open, search, freeze or close accounts | account, account_type, branch |
| **Transactions** | Post deposits and withdrawals; audit log | transaction, v_transaction_audit |
| **Transfers** | Atomic transfer; beneficiaries | transfer, beneficiary, sp_transfer |
| **Statements** | Statement for any period with running balance | statement, v_account_statement |
| **Loans** | Create loan with EMI schedule; repay instalments | loan, loan_schedule, repayment, sp_loan_repayment |
| **Rule Tester** | Try illegal operations; see them refused and rolled back | transaction, transfer, account |

# Roles and Permissions

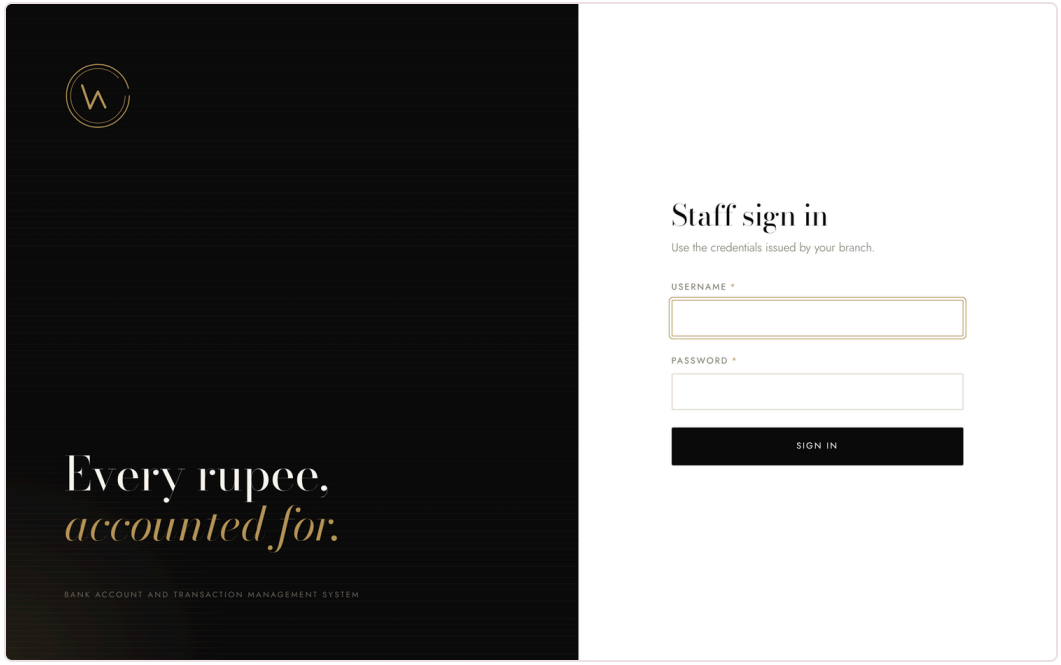
| CAPABILITY | TELLER | LOAN OFFICER | MANAGER |
|---|---|---|---|
| **View dashboard, customers, accounts, loans, statements** | ✓ | ✓ | ✓ |
| **Register customers · open accounts** | ✓ | — | ✓ |
| **Post deposits and withdrawals · transfers · beneficiaries** | ✓ | — | ✓ |
| **Create loans** | — | ✓ | ✓ |
| **Repay a loan instalment** | ✓ | ✓ | ✓ |
| **Freeze or close an account** | — | — | ✓ |
| **Transaction audit log** | — | — | ✓ |

Enforced on the server for every request. A denied action returns HTTP 403 and the screen hides the button.

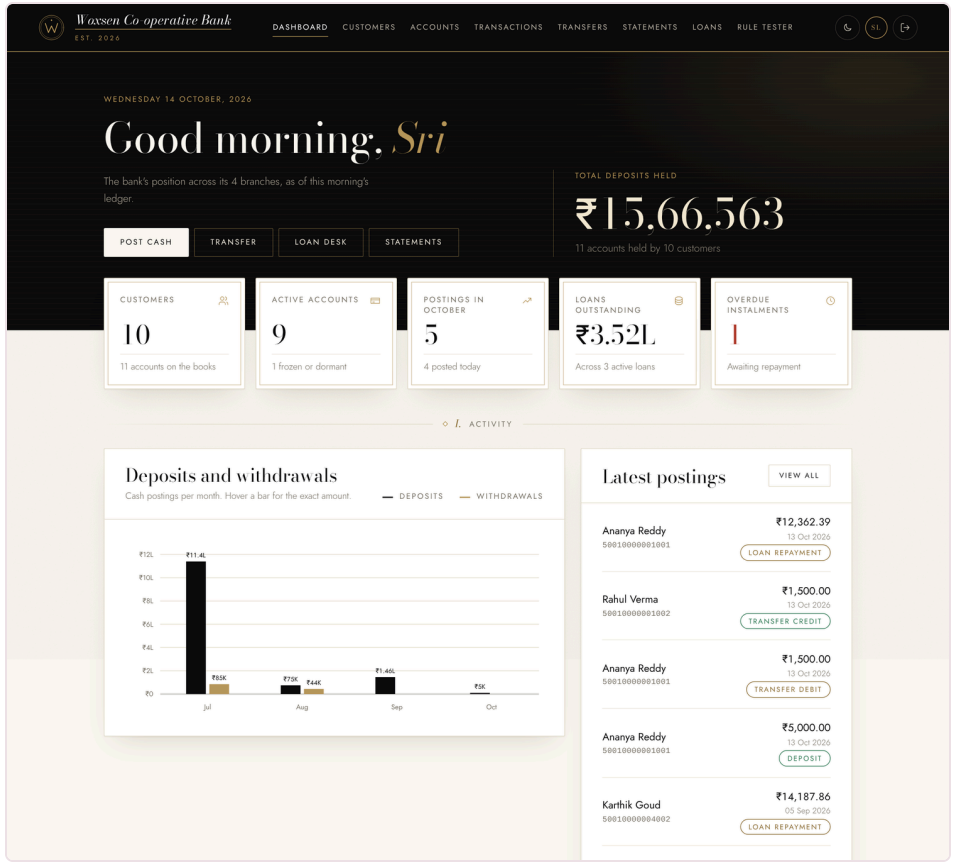
# Business Rules: Enforced by the Database

| RULE | WHERE ENFORCED | WHAT THE APP SHOWS |
|---|---|---|
| **Amount must be positive** | BEFORE INSERT trigger + CHECK | 422 “Transaction amount must be positive” |
| **No overdraft below minimum balance** | Trigger, with the account row locked | 422 “Insufficient balance (minimum balance rule)” |
| **Only ACTIVE accounts accept postings** | Trigger | 422 “Account is not ACTIVE” |
| **Transfer legs must balance; no same-account transfer** | Trigger + sp_transfer | 422 “Unbalanced transfer” / “Cannot transfer to the same account” |
| **Unique account number, email, ID proof** | UNIQUE constraints | 409 “Duplicate entry” |
| **Instalment can be paid only once** | sp_loan_repayment + app guard | 409 “Instalment is already fully paid” |
| **Malformed input** | Shape checks before the query | 400 with the field name |

# Screens — Sign-in and Manager Dashboard

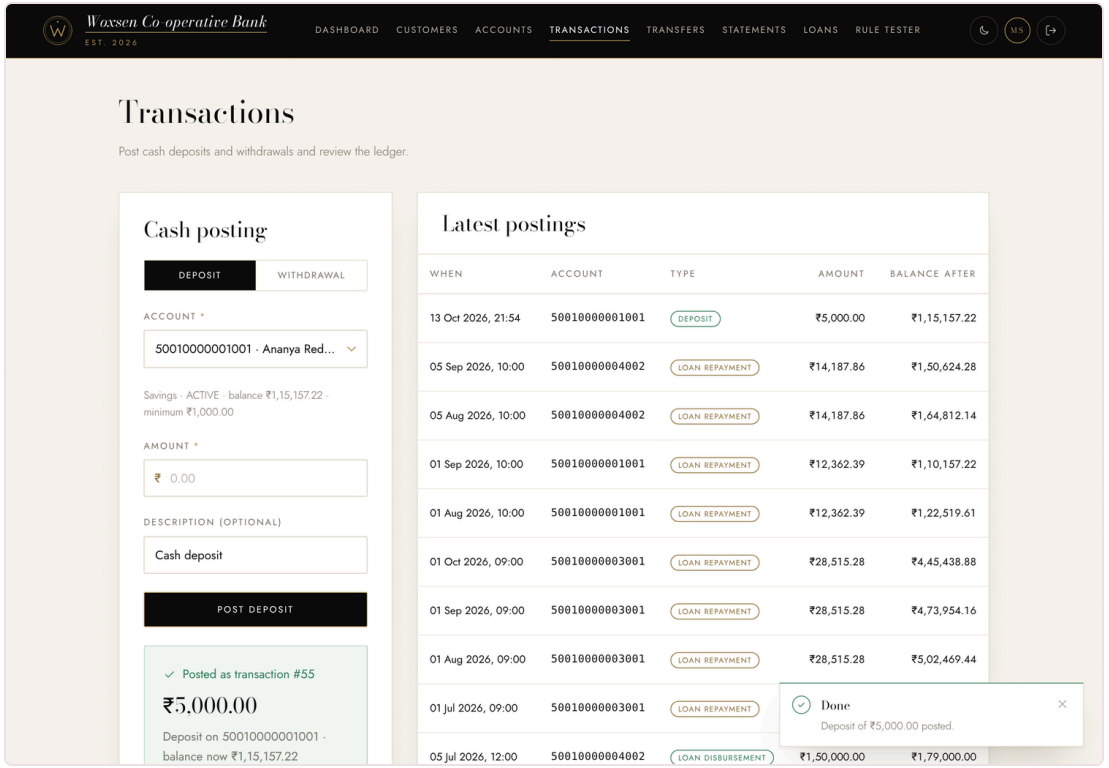



Staff sign-in

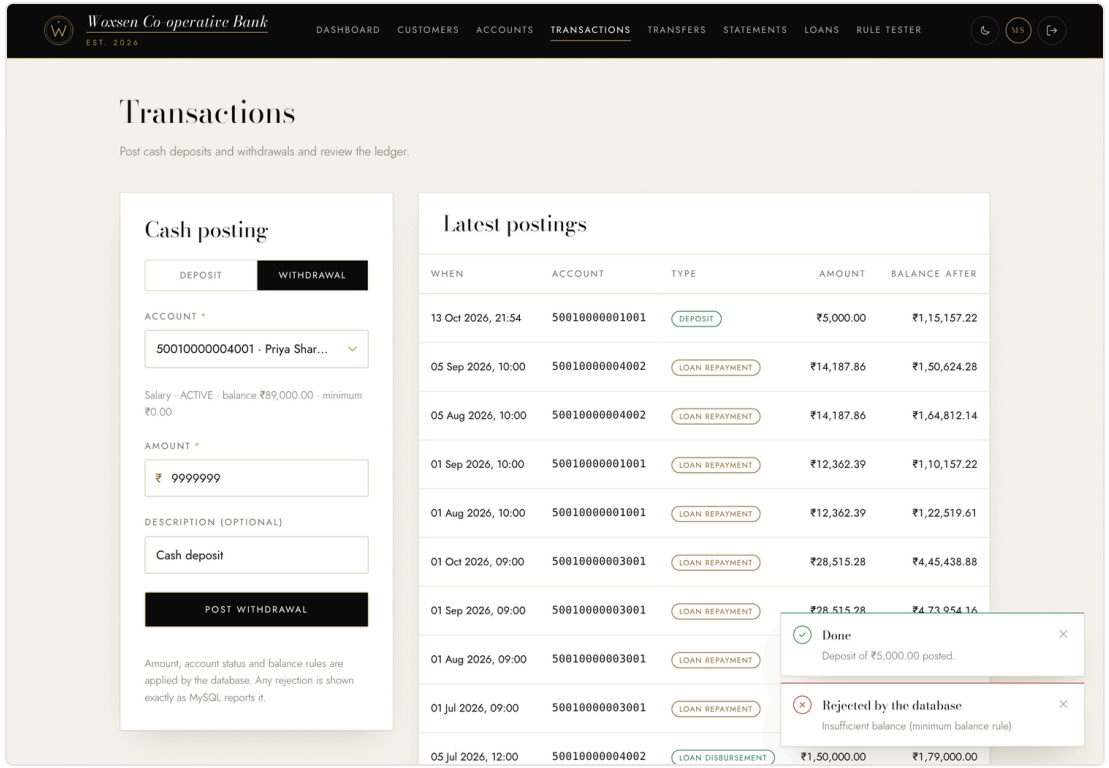



Dashboard: live totals, deposits and withdrawals, branch summary, latest postings

# Screens — Deposit Posted, Overdraw Refused

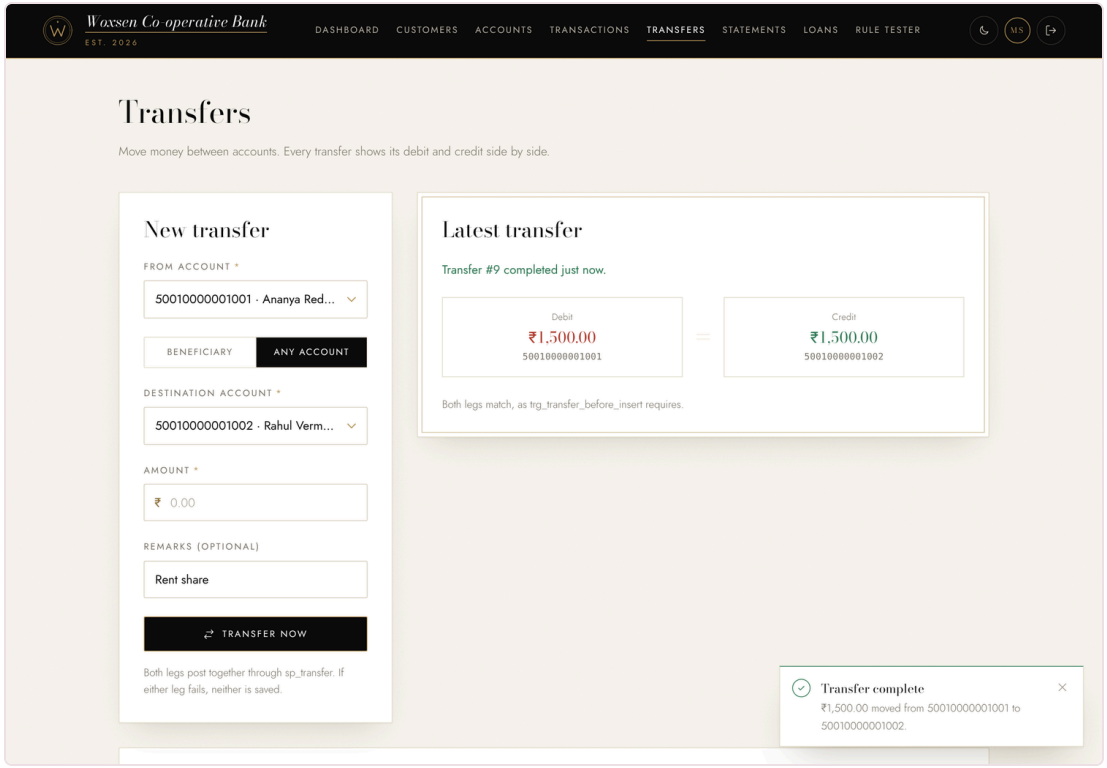



Deposit of INR 5,000: posting created and balance updated by the trigger

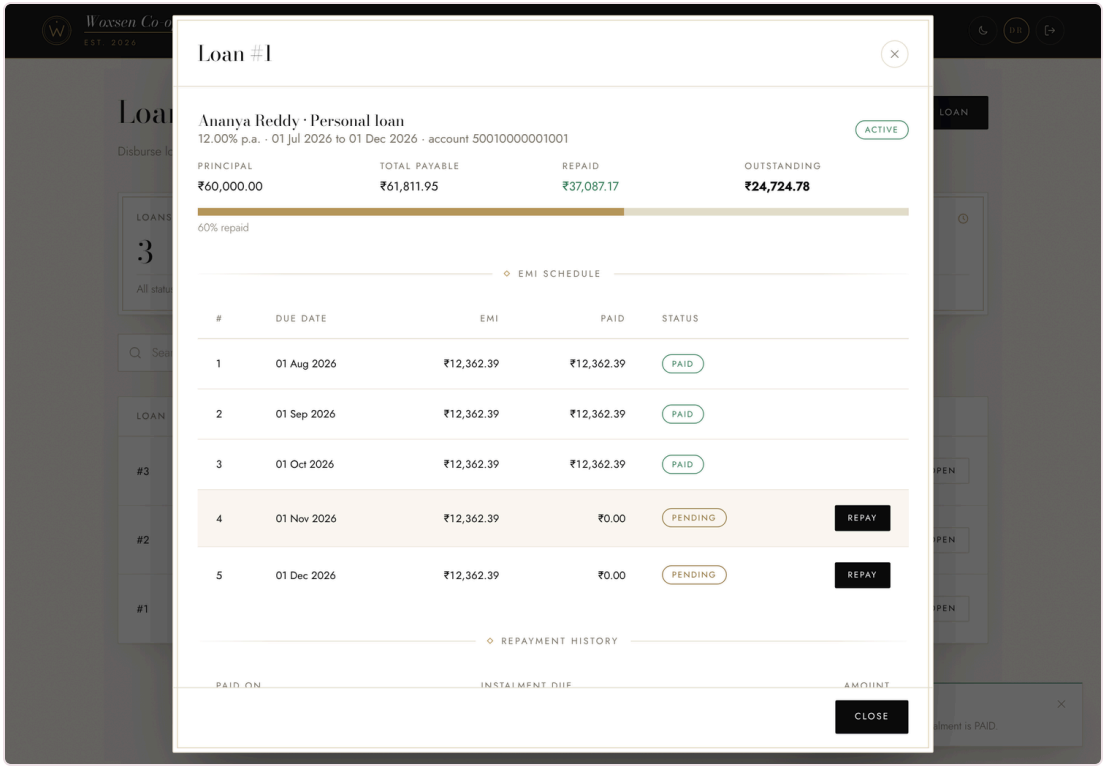



Withdrawal beyond the balance: the database rejects it, balance unchanged

# Screens — Atomic Transfer, Loan Repayment

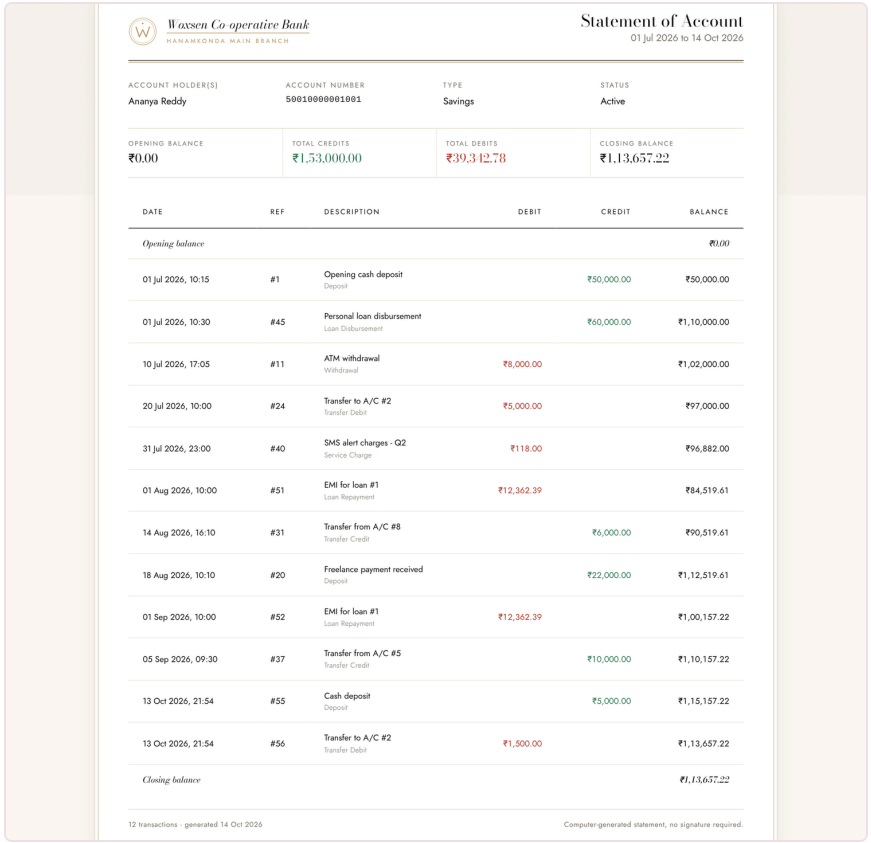



Transfer: debit and credit posted together, both balances move




Loan schedule after an EMI repayment: instalment marked PAID

# Screens — Statement and Rule Tester



Woxsen Co-operative Bank
HANAMKONDA MAIN BRANCH

Statement of Account
01 Jul 2026 to 14 Oct 2026

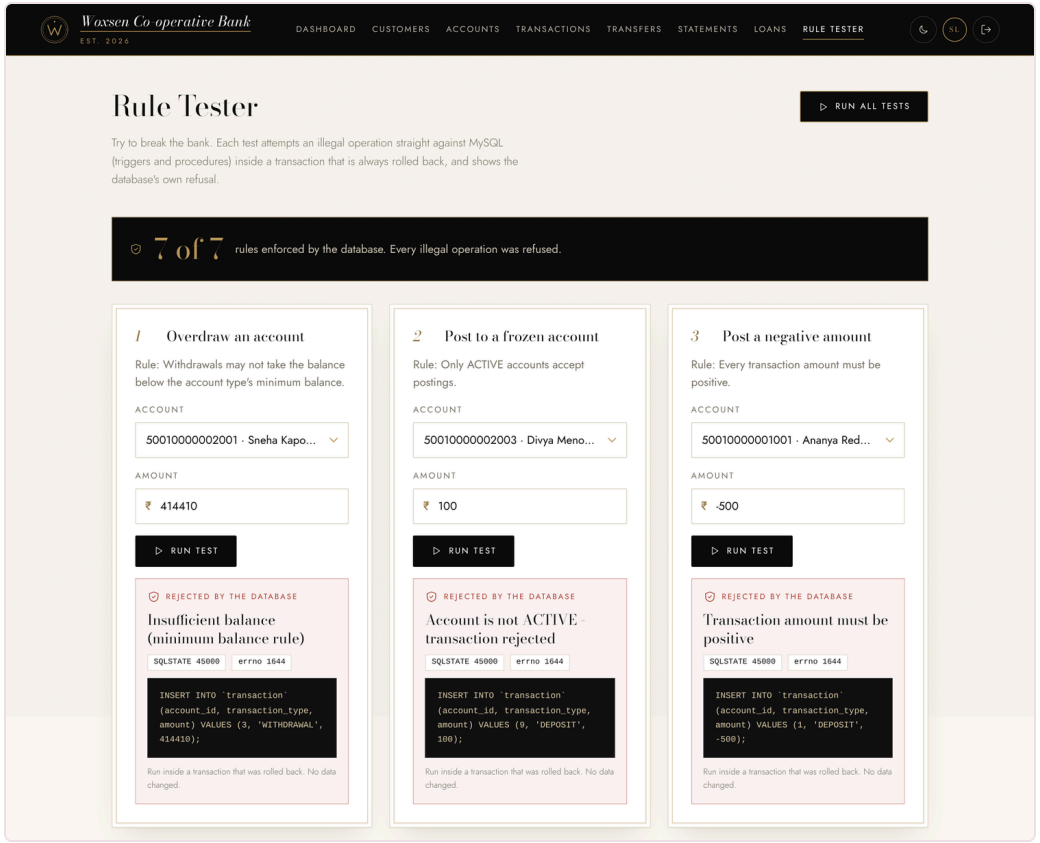
| ACCOUNT HOLDER(S) | ACCOUNT NUMBER | TYPE | STATUS |
|---|---|---|---|
| Ananya Reddy | 50010000001001 | Savings | Active |

| OPENING BALANCE | TOTAL CREDITS | TOTAL DEBITS | CLOSING BALANCE |
|---|---|---|---|
| ₹0.00 | ₹1,53,000.00 | ₹39,342.78 | ₹1,13,657.22 |

| DATE | REF | DESCRIPTION | DEBIT | CREDIT | BALANCE |
|---|---|---|---|---|---|
| *Opening balance* | | | | | *₹0.00* |
| 01 Jul 2026, 10:15 | #1 | Opening cash deposit<br>Deposit | | ₹50,000.00 | ₹50,000.00 |
| 01 Jul 2026, 10:30 | #45 | Personal loan disbursement<br>Loan Disbursement | | ₹60,000.00 | ₹1,10,000.00 |
| 10 Jul 2026, 17:05 | #11 | ATM withdrawal<br>Withdrawal | ₹8,000.00 | | ₹1,02,000.00 |
| 20 Jul 2026, 10:00 | #24 | Transfer to A/C #2<br>Transfer Debit | ₹5,000.00 | | ₹97,000.00 |
| 31 Jul 2026, 23:00 | #40 | SMS alert charges - Q2<br>Service Charge | ₹118.00 | | ₹96,882.00 |
| 01 Aug 2026, 10:00 | #51 | EMI for loan #1<br>Loan Repayment | ₹12,362.39 | | ₹84,519.61 |
| 14 Aug 2026, 16:10 | #31 | Transfer from A/C #8<br>Transfer Credit | | ₹6,000.00 | ₹90,519.61 |
| 18 Aug 2026, 10:10 | #20 | Freelance payment received<br>Deposit | | ₹22,000.00 | ₹1,12,519.61 |
| 01 Sep 2026, 10:00 | #52 | EMI for loan #1<br>Loan Repayment | ₹12,362.39 | | ₹1,00,157.22 |
| 05 Sep 2026, 09:30 | #37 | Transfer from A/C #5<br>Transfer Credit | | ₹10,000.00 | ₹1,10,157.22 |
| 13 Oct 2026, 21:54 | #55 | Cash deposit<br>Deposit | | ₹5,000.00 | ₹1,15,157.22 |
| 13 Oct 2026, 21:54 | #56 | Transfer to A/C #2<br>Transfer Debit | ₹1,500.00 | | ₹1,13,657.22 |
| *Closing balance* | | | | | *₹1,13,657.22* |

12 transactions · generated 14 Oct 2026 — Computer-generated statement, no signature required.

Account statement with running balance



Woxsen Co-operative Bank
EST. 2026

DASHBOARD · CUSTOMERS · ACCOUNTS · TRANSACTIONS · TRANSFERS · STATEMENTS · LOANS · RULE TESTER

Rule Tester

RUN ALL TESTS

Try to break the bank. Each test attempts an illegal operation straight against MySQL (triggers and procedures) inside a transaction that is always rolled back, and shows the database's own refusal.

7 of 7 rules enforced by the database. Every illegal operation was refused.

1 Overdraw an account

Rule: Withdrawals may not take the balance below the account type's minimum balance.

ACCOUNT: 50010000002001 · Sneha Kapo...
AMOUNT: ₹ 414410
RUN TEST

REJECTED BY THE DATABASE
Insufficient balance (minimum balance rule)
SQLSTATE 45000 · errno 1644

```
INSERT INTO `transaction`
(account_id, transaction_type,
amount) VALUES (3, 'WITHDRAWAL',
414410);
```

Run inside a transaction that was rolled back. No data changed.

2 Post to a frozen account

Rule: Only ACTIVE accounts accept postings.

ACCOUNT: 50010000002003 · Divya Meno...
AMOUNT: ₹ 100
RUN TEST

REJECTED BY THE DATABASE
Account is not ACTIVE - transaction rejected
SQLSTATE 45000 · errno 1644

```
INSERT INTO `transaction`
(account_id, transaction_type,
amount) VALUES (9, 'DEPOSIT',
100);
```

Run inside a transaction that was rolled back. No data changed.

3 Post a negative amount

Rule: Every transaction amount must be positive.

ACCOUNT: 50010000001001 · Ananya Red...
AMOUNT: ₹ -500
RUN TEST

REJECTED BY THE DATABASE
Transaction amount must be positive
SQLSTATE 45000 · errno 1644

```
INSERT INTO `transaction`
(account_id, transaction_type,
amount) VALUES (1, 'DEPOSIT',
-500);
```

Run inside a transaction that was rolled back. No data changed.

Rule Tester: illegal operations are refused and rolled back

# Proof in MySQL — Triggers Reject Illegal Postings

```
mysql — bank_db

mysql> SELECT account_id, status, balance FROM account WHERE account_id IN (5,9);
+------------+--------+----------+
| account_id | status | balance  |
+------------+--------+----------+
|          5 | ACTIVE | 89000.00 |
|          9 | FROZEN | 15000.00 |
+------------+--------+----------+

mysql> INSERT INTO `transaction` (account_id, transaction_type, amount, description) VALUES (5,'WITHDRAWAL',9999999,'overdraw');
--------------
INSERT INTO `transaction` (account_id, transaction_type, amount, description) VALUES (5,'WITHDRAWAL',9999999,'overdraw')
--------------

ERROR 1644 (45000): Insufficient balance (minimum balance rule)

mysql> INSERT INTO `transaction` (account_id, transaction_type, amount, description) VALUES (9,'DEPOSIT',1000,'frozen account');
--------------
INSERT INTO `transaction` (account_id, transaction_type, amount, description) VALUES (9,'DEPOSIT',1000,'frozen account')
--------------

ERROR 1644 (45000): Account is not ACTIVE - transaction rejected

mysql> INSERT INTO `transaction` (account_id, transaction_type, amount, description) VALUES (1,'DEPOSIT',-500,'negative');
--------------
INSERT INTO `transaction` (account_id, transaction_type, amount, description) VALUES (1,'DEPOSIT',-500,'negative')
--------------

ERROR 1644 (45000): Transaction amount must be positive

mysql> SELECT account_id, status, balance FROM account WHERE account_id IN (5,9); -- unchanged
+------------+--------+----------+
| account_id | status | balance  |
+------------+--------+----------+
|          5 | ACTIVE | 89000.00 |
|          9 | FROZEN | 15000.00 |
+------------+--------+----------+
```

Overdraw, frozen-account posting and negative amount are rejected; balances unchanged

```
mysql — bank_db

mysql> SHOW FULL TABLES FROM bank_db WHERE Table_type = 'BASE TABLE';
+-------------------+------------+
| Tables_in_bank_db | Table_type |
+-------------------+------------+
| account           | BASE TABLE |
| account_holder    | BASE TABLE |
| account_type      | BASE TABLE |
| beneficiary       | BASE TABLE |
| branch            | BASE TABLE |
| customer          | BASE TABLE |
| loan              | BASE TABLE |
| loan_repayment    | BASE TABLE |
| loan_schedule     | BASE TABLE |
| repayment         | BASE TABLE |
| service_charge    | BASE TABLE |
| statement         | BASE TABLE |
| transaction       | BASE TABLE |
| transfer          | BASE TABLE |
+-------------------+------------+
```

The 14 base tables of bank_db

- The same rules hold when SQL is run directly in MySQL, outside the application.
- Rejected statements leave no partial rows.

# Testing and Results

| 16 | 15 / 15 | 0 | 0 |
|---|---|---|---|
| Test cases, all Pass | End-to-end tests | Balance mismatches | Unbalanced transfers |

| TEST | INPUT | RESULT |
|---|---|---|
| **TC-04 Deposit** | INR 5,000 to account …1001 | Posted; balance 1,10,157.22 → 1,15,157.22 |
| **TC-05 Overdraw** | INR 99,99,999 withdrawal | 422 Insufficient balance; balance unchanged |
| **TC-07 / 08 Frozen, closed** | Deposit INR 1,000 | 422 Account is not ACTIVE |
| **TC-09 Transfer** | INR 1,500 …1001 → …1002 | Balanced; both balances moved |
| **TC-12 Loan repayment** | Pay an instalment | Marked PAID; outstanding reduced |
| **TC-16 Ledger integrity** | Compare balance with last ledger entry | 54 of 54 transactions agree |

Positive and negative cases; actual results were taken from the running application and the live database.

# Conclusion, Limitations and Future Work

## CONCLUSION

- A normalized 14-table banking database with rules enforced where the data lives.
- A role-based web application that cannot bypass those rules.
- Every claim backed by a test, a screenshot or a query result.

## LIMITATIONS AND FUTURE WORK

- Interest on savings and fixed deposits is not posted automatically; add a scheduled job.
- External transfers (NEFT/UPI) need a payment gateway.
- Customer self-service login with OTP, and PDF statements.
- Partition the transaction ledger by month for large volumes.

# Thank You

Questions and technical discussion

Live demo: srilakshmireddy-byte.github.io/bank-dbms-pbl

Code: github.com/srilakshmireddy-byte/bank-dbms-pbl